```
qnext2 = torch.zeros(N2)

act_means1 = init_mean_field(N1,act_space)                    5
act_means2 = init_mean_field(N2,act_space)

rewards1 = torch.zeros(N1)                                    6
rewards2 = torch.zeros(N2)

losses1 = []
losses2 = []
```

- ***1*** Stores the number of agents on each team
- ***2*** Initializes the actions for all the agents
- ***3*** Creates replay buffer using a deque data structure
- ***4*** Creates tensors to store the Q(s') values, where s' is the next state
- ***5*** Initializes the mean fields for each agent
- ***6*** Creates tensors to store the rewards for each agent

The variables in listing 9.19 allow us to keep track of the actions (integers), mean field action vectors, rewards, and next state Q values for each agent so that we can package these into experiences and add them into the experience replay system. In listing 9.20 we define a function to take actions on behalf of a particular team of agents and another function to store experiences in the replay buffer.

Listing 9.20. Taking a team step and adding to the replay

```
def team_step(team,param,acts,layers):
    obs = env.get_observation(team)
1
    ids = env.get_agent_id(team)
2
    obs_small = torch.from_numpy(obs[0][:,:,:,[1,4]])
3
    agent_pos = env.get_pos(team)
4
    acts, act_means, qvals = infer_acts(obs_small,\
                                        param,layers,agent_pos,acts)
5
    return acts, act_means, qvals, obs_small, ids

def add_to_replay(replay,obs_small, acts,rewards,act_means,qnext):
6
    for j in range(rewards.shape[0]):
7
```